PRÁCTICA OPENCODE UT1. DOCUMENTAR Y VERSIONAR CON UN AGENTE

Antes de empezar

opencode instalado (ver la **guía de instalación**) y el proyecto en un repositorio Git, que es tu botón de deshacer. Comprueba con /models qué modelo tienes activo: con los modelos gratuitos tendrás que ser más explícito en los prompts y cometerán más errores, lo que viene bien para lo que se pide aquí. Necesitas además Node.js 18 o superior (node --version) y el proyecto gestor-tareas de la carpeta recursos de esta práctica, copiado a tu carpeta de trabajo.

- Trabaja en **modo Plan** (tecla Tab) hasta que el plan te convenza; solo entonces pasa a **Build**.
- Revisa cada cambio antes de aceptarlo (git diff). Si no sabes explicar una línea, no la aceptes.
- **Verifica ejecutando**: la evidencia es la salida del comando, no la respuesta del agente.
- Anota cada prompt en prompts.md: qué pediste, qué hizo y qué corregiste.
- Nada de contraseñas ni datos personales en el chat: los modelos gratuitos son de prueba y no admiten información confidencial.
- Si el modelo se atasca, da vueltas o va muy lento, divide la petición en pasos o cambia de modelo con /models.

Ejercicios

1) **Primer contacto.** Copia gestor-tareas a tu carpeta de trabajo, inicializa un repositorio Git y haz un commit con el proyecto tal cual («Proyecto de partida»). Arranca opencode dentro de la carpeta y comprueba el modelo activo con /models. En **modo Plan** pídele: «explica qué hace este proyecto, qué ficheros lo componen y cómo se ejecuta». Captura la respuesta y **comprueba tú** que los ficheros y los comandos que cita existen de verdad. (0,5p)

Respuesta:

Copié el proyecto a mi carpeta de trabajo, creé el repositorio con git init -b main y confirmé el primer commit «Proyecto de partida». Con /models comprobé que el modelo activo es DeepSeek V4.1 Flash Free.

En modo Plan pedí que explicara el proyecto. En resumen: es una aplicación de consola en JavaScript (CommonJS) sin dependencias que guarda las tareas en datos/tareas.json; la componen package.json (scripts),

src/cli.js (interfaz: anadir, completar, eliminar, listar y resumen), src/tareas.js (lógica y persistencia) y datos/tareas.json (array de tareas con id, texto, prioridad y hecha). Se ejecuta con:

```
node src/cli.js <orden>   o   npm start -- <orden>
```

Lo comprobé yo mismo ejecutando los comandos:

```
$ ls -R  ->  datos/tareas.json, package.json, src/cli.js, src/tareas.js
$ node --version
v22.22.1
$ node src/cli.js listar
[1] (p1) Preparar el entorno de despliegue
[3] (p2) Configurar el repositorio remoto
[2] (p3) Escribir el README del proyecto
$ node src/cli.js resumen
{ total: 3, hechas: 0, pendientes: 3 }
```

De paso vi que completar no marcaba la tarea esperada; lo confirmé en el ejercicio 6.

2) **AGENTS.md.** Ejecuta /init para que genere el fichero AGENTS.md. Léelo entero, **corrige lo que no sea cierto** y añade al menos cuatro normas del proyecto: documentación y commits en español, comentarios JSDoc en src/, mensajes de commit «Añade…/Corrige…», y prohibición de subir cambios al remoto. Explica para qué sirve este fichero y por qué se versiona. (1p)

Respuesta:

Le pedí que generara AGENTS.md (equivale a /init) y lo revisé entero. Se inventó una carpeta test/ que no existe; lo comprobé y la quité:

```
$ ls -d test
ls: no se puede acceder a 'test': No existe el archivo o directorio
```

Añadí las cuatro normas: documentación y commits en español, JSDoc en todas las funciones de src/, mensajes de commit «Añade…/Corrige…» y prohibición de subir cambios al remoto.

¿Para qué sirve AGENTS.md? Es el fichero que da contexto a los agentes que trabajen en el repositorio (qué es el proyecto, cómo se ejecuta y qué normas hay). ¿Por qué se versiona? Porque es parte del proyecto: evoluciona con él, se revisa en los diffs como cualquier otro fichero y todos ven las normas vigentes.

3) **Permisos.** Crea el fichero opencode.json del proyecto para que el agente: pregunte antes de ejecutar órdenes del sistema, tenga **prohibido** git push y rm, no pueda leer .env ni escribir fuera de src/, test/ y docs/, y no pueda compartir la sesión. Demuestra con una captura que un git push pedido al agente queda bloqueado. (1p)

Respuesta:

Creé opencode.json con esta política: preguntar antes de ejecutar cualquier orden (shell:* -> ask), prohibir git push y rm, no poder leer ficheros .env, no poder escribir en general (edit:* -> deny) salvo en src/, test/ y docs/ (más README.md, AGENTS.md, prompts.md y opencode.json, que la práctica obliga a editar) y la sesión sin posibilidad de compartirse.

Lo demostré pidiéndole un git push: quedó bloqueado antes de ejecutarse.

```
$ git push origin main
Permission denied: shell
```

Igual al intentar escribir un fichero fuera de las carpetas permitidas: Permission denied: edit.



Captura real: el intento de git push queda bloqueado por la política de permisos.

4) **Documentación del código.** Pide al agente que documente con **JSDoc** las seis funciones de src/tareas.js (@param, @returns, @throws y un @example). Revisa una por una: la descripción debe coincidir con lo que hace el código. Anota al menos **una descripción incorrecta o inventada** y corrígela. (1,5p)

Respuesta:

Pedí el JSDoc de las seis funciones de src/tareas.js (@param, @returns, @throws y un @example) y lo revisé una por una comparándolo con el código.

Encontré una descripción incorrecta en resumen(): decía que devolvía un número, pero devuelve un objeto. La corregí:

```
(antes)    @returns {number} Número de tareas que aún no están hechas.
(después)  @returns {{total: number, hechas: number, pendientes: number}}
```

También anoté que la descripción de completar() hablaba del id y el código usaba el índice del array; eso lo arreglé en el ejercicio 6.

5) **README.** Pídele un README.md con: título, descripción, requisitos, instalación, uso con un ejemplo por cada orden de la CLI, estructura de carpetas, formato de datos/tareas.json, autoría y licencia. **Ejecuta todos los comandos del README**: corrige los que no funcionen o den una salida distinta a la documentada y deja constancia de la corrección. (1,5p)

Respuesta:

Escribí README.md con todas las secciones (título, descripción, requisitos, instalación, uso con un ejemplo de cada orden, estructura de carpetas, formato de datos/tareas.json, autoría y licencia) y después ejecuté TODOS los comandos del documento. Encontré tres cosas mal:

1) anadir: decía «Tarea 4 creada con prioridad 2» y la salida real es «Tarea creada con id 4».

2) eliminar: decía «Eliminada; quedan 3 tareas» y la salida real es «Eliminada».

3) npm run listar no existe («npm error Missing script: listar»); lo quité y dejé el correcto npm start -- listar.

Salidas reales con las que comprobé:

```
$ node src/cli.js anadir "Comprar pan"
Tarea creada con id 4
$ node src/cli.js eliminar 4
Eliminada
```

Con completar 1 vi que marcaba la tarea 2 en lugar de la 1: no era un error del README (documentaba lo correcto), era un fallo del código, que resolví en el ejercicio 6. Dejé el README corregido tras ejecutarlo todo.

6) **El error.** El proyecto tiene un fallo: `node src/cli.js completar 1` no marca la tarea que debería. Sin decirle cuál es el fallo, pide al agente que escriba en `test/` una prueba con `node:test` que lo demuestre y ejecútala con `npm test` (debe **fallar**). Solo después, pídele que corrija `src/tareas.js` y vuelve a ejecutar las pruebas (deben **pasar**). Adjunta las dos salidas y explica en qué consistía el error. (2p)

Respuesta:

El fallo estaba en completar(id): usaba el argumento como índice del array (tareas[id]) en vez de buscar por id. Como los id empiezan en 1 y los índices en 0, completar 1 marcaba la segunda tarea. Escribí la prueba test/tareas.test.js con node:test (haciendo copia de seguridad de datos/tareas.json) y falló como debía:

```
not ok 1 - completar(1) marca como hecha la tarea con id 1 (y solo esa)
  error: la tarea con id 1 debería estar hecha
  false !== true
# tests 1   # pass 0   # fail 1
```

Corregí la función:

```
-  tareas[id].hecha = true;
+  const tarea = tareas.find((t) => t.id === id);
+  if (!tarea) throw new Error(`No existe ninguna tarea con el id ${id}`);
+  tarea.hecha = true;
```

Y ahora la prueba pasa:

```
ok 1 - completar(1) marca como hecha la tarea con id 1 (y solo esa)
# tests 1   # pass 1   # fail 0
```

Comprobado también con el comando real: node src/cli.js completar 1 devuelve la tarea con id 1.

7) **Git con supervisión.** Pide al agente que prepare los cambios en **commits separados** (documentación, pruebas y corrección) con mensajes en español. Antes de cada commit revisa git diff --staged. Después, crea con él docs/control-de-versiones.md: instalación y configuración de Git usada, flujo de trabajo, convención de mensajes y cómo revisar lo que hace un agente antes de confirmarlo. Verifica el historial con git log --oneline --graph. (1,5p)

Respuesta:

Hice los commits separados por tema y en español («Añade…/Corrige…»), revisando git diff --staged antes de confirmar cada uno:

```
852f7f8 Añade prueba de completar() con node:test
1d51aea Corrige completar() para localizar la tarea por su id
47c1361 Añade documentación JSDoc a las funciones de src/tareas.js
```

Configuración usada: git 2.53.0 con la identidad RD <rd@example.com>; sin remoto configurado (las subidas están prohibidas).

También creé docs/control-de-versiones.md con la instalación y configuración, el flujo de trabajo (un tema = un commit; revisar el diff y ejecutar las pruebas antes de confirmar), la convención de mensajes y una lista para revisar a un agente: git status y git diff, git diff --staged, ejecutar la verificación real (npm test y los ejemplos del README), comprobar el mensaje y apoyarse en los permisos de opencode.json.

8) **Comparación de formatos.** Convierte el README.md a HTML y a PDF (extensión Markdown PDF de VS Code, Pandoc o el propio navegador) y añade al README una tabla que compare los tres formatos: ventajas, inconvenientes y cuándo usar cada uno. (0,5p)

Respuesta:

Convertí el README a HTML y PDF con un script que dejé en docs/ (bash docs/convertir-readme.sh: Markdown -> HTML con markdown-it y HTML -> PDF con LibreOffice). Se generan docs/README.html y docs/README.pdf (5 páginas). Añadí al README la tabla comparativa:

- Markdown: fácil de escribir, editar y revisar en Git; no controla el diseño final; es la fuente de la documentación.

- HTML: presentación controlada con CSS y se abre en cualquier navegador; más verboso y con peores diffs; para publicar en web.

- PDF: presentación fija, cómodo de entregar e imprimir; binario, no se revisa bien en Git; para entregas.

HTML y PDF son formatos generados: se regeneran desde el Markdown para que no queden desactualizados.

9) **Balance.** Responde en prompts.md: ¿cuántas correcciones tuviste que hacerle? ¿qué parte te ahorró tiempo de verdad? ¿qué habrías entregado mal si no lo hubieras comprobado? (0,5p)

Respuesta:

Mantuve el registro de prompts en prompts.md (qué pedí, qué hizo el agente y qué corregí; 11 filas al final). Balance:

- ¿Cuántas correcciones hice? Seis: la carpeta test/ inventada en AGENTS.md, la descripción incorrecta de resumen() en el JSDoc, dos salidas mal documentadas en el README (anadir y eliminar), el atajo inexistente npm run listar y la detección del fallo de completar (esta era del código, no de la documentación).

- ¿Qué me ahorró tiempo de verdad? Los borradores completos (AGENTS.md, el JSDoc, el README y la guía de Git) y el trabajo mecánico: la prueba inicial, los mensajes de commit y la conversión a HTML/PDF.

- ¿Qué habría entregado mal sin comprobarlo? Un AGENTS.md con una carpeta que no existe, un JSDoc falso, un README con salidas y un comando equivocados y, sobre todo, un completar() que marca la tarea equivocada: el código «parecía» correcto hasta ejecutar la prueba.

Registro de prompts

Mantén durante toda la práctica el fichero prompts.md con una tabla como esta (una fila por prompt importante):

N°

Prompt (resumido)

Qué hizo el agente

Qué corregí yo